

Gemini CLI 活用ガイド (学生参加者向け)

このハッカソンでは、開発パートナーとして **Gemini CLI** を活用できます。AIを賢く使いこなし、効率的に強力なポーカープレイヤーを構築しましょう。

目次

- 1. AIの主な利用方法
 - 2. コーディングエージェント使用時の注意点
 - 3. 具体的な質問例 (カテゴリー別)
 - 🔍 プログラムの検索・仕様の確認
 - 📖 プログラムの解説・ルールの確認
 - 💡 戦略の実装・レビュー依頼
 - 🛠 デバッグ・テスト
-

0. Gemini CLI の起動方法

プロジェクトのルートディレクトリで以下のコマンドを実行してください。

```
cd [プロジェクトのディレクトリ] # パスは各自の環境に合わせて変更してください
gemini
```

※ 初回起動時や、まだ **gemini** コマンドが使えない場合は、**環境構築ガイド**を参照してセットアップを完了させてください。

1. AIの主な利用方法

Gemini CLIは、単なるチャットツールではなく、プロジェクトのファイルを直接読み書きできる「コーディングエージェント」です。以下のようなシーンで活用してください。

- **プログラムの検索**: 「〇〇の処理はどのファイルに書いてある？」といった探索。
 - **プログラムの解説**: 既存のコードやユーティリティ関数の役割を理解する。
 - **ルールの確認**: **docs/rule.md** などのドキュメントに基づいたルールの把握。
 - **戦略の実装依頼**: 自分が考えたロジックをコード (TypeScript) に変換してもらう。
 - **戦略へのレビュー依頼**: 実装したアルゴリズムの弱点や改善案を相談する。
 - **デバッグ**: エラーメッセージの原因特定と修正案の提示。
 - **仕様の確認**: 関数の引数や戻り値の型、利用可能な定数の確認。
-

2. コーディングエージェント使用時の注意点

AIは強力ですが、万能ではありません。以下のポイントに注意してください。

△ 必ず動作確認をすること

AIは時として、存在しない関数を呼び出したり、もっともらしい嘘（**ハルシネーション**）をつくことがあります。「AIが書いたから正しい」と思い込まず、必ずブラウザ上で意図した通りに動くか自分の目で確認してください。

💬 正確なアウトプットを出させるコツ

曖昧な質問には、曖昧な回答しか返ってきません。

- **文脈（Context）を伝える**: 「どのファイルを編集しているか」「どの型定義を参照しているか」を明示すると精度が上がります。
- **具体的・段階的に聞く**: いきなり「最強のAIを作って」と言うのではなく、「まずは役を判定する」「次に賭け金額を決める」とステップを分けて依頼しましょう。
- **制約を与える**: 「既存の関数を再利用して」「この変数だけを使って」といった制約が、精度の高い回答を引き出します。

📊 利用制限とトークンの意識

AIとのやり取りには「トークン（文字数のようなもの）」を消費します。

- **トークンの確認方法**: 対話中に `/stats model` を入力し、「Model usage」の項目で現在の消費量や残り（クォータ）を確認できます。
- **制限に達した場合**: 一時的にメッセージの送信やAIによる操作ができなくなります。その場合は、**現在のプログラムをチームメンバーに共有し、別のメンバーが Gemini CLI を起動して開発を引き継いでください。**
- **履歴の蓄積**: 会話が長くなりすぎるとAIの記憶が混濁し、レスポンスが悪くなることがあります。大きなタスクが終わったら、一度目的を整理し直すのがスマートです。

3. 具体的な質問例（カテゴリー別）

🔍 プログラムの検索・仕様の確認

- 「自分が編集してよいプレイヤーコードの場所と、編集すべき関数名を教えてください。」
- 「ベットフェーズの関数の引数に含まれる `playerStatus` には、具体的にどんなプロパティ（残りポイント、現在の役など）が含まれていますか？」
- 「手札の強さを判定するために使える既存の関数を `src/utils/` から探して、その使い方を説明してください。」

📖 プログラムの解説・ルールの確認

- 「`src/utils/hand.ts` 中にある役判定ロジックを、初心者にもわかるように1行ずつ解説してください。」
- 「ルール上、1ラウンドで最大何回までレイズが可能ですか？ `docs/rule.md` から探してください。」

💡 戦略の実装・レビュー依頼

- **実装依頼**: 「『もし自分の手札がワンペア以上で、かつポットに積まれたポイントが自分の所持金の1/4を超えているならレイズする。それ以外はコールする』というロジックを、ベットフェーズの関数に実装してください。」

- **レビュー依頼:** 「私は『残りラウンドが10回以下の時だけ、手札に関わらず全額ベット（オールイン）する』という戦略を考えました。この戦略のメリットと、ルール上のリスクを分析してください。」

デバッグ・テスト

- 「（エラーメッセージを貼り付けて）このエラーが出ました。自分のコードのどこに型定義のミスがあるか、`src/schema/` の定義と比較して修正案を出してください。」
- 「自分が作成したプレイヤー（team1）だけをテストするための Jest の実行コマンドを教えてください。」